MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 1

Software Design

Author: *Cristina Adriana Ciors*
Group: *30433*

Faculty of Automation and Computer Science

March *2026*

Contents

1 Problem definition

The core objective of this project is to implement a simple but well-structured application using object-oriented principles and a layered architecture that manages a set of domain entities. The chosen domain for this system is a Trading Card Game (TCG) Shop. The application provides a complete graphical interface that handles inventory management, allowing users to browse, search, and filter TCG products and categories

To ensure proper access control, the application must support multiple user roles and allow users to interact with the system according to their permissions. The system implements three distinct roles : Visitors who can view public information but cannot modify data , a secondary domain-specific role (Clients) representing authenticated shoppers , and Administrators who have full access to manage the system and perform CRUD operations on all entities. The application must follow a MVC (Model-View-Controller) architecture , strictly separating the data access, business logic, and presentation layers.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

- The system must allow users to securely authenticate and register for a new account to access role-specific functionalities.
- The system must enable users with Administrator privileges to perform full CRUD (Create, Read, Update, Delete) operations on the core domain entities, specifically inventory items and categories.
- The system must allow authenticated Clients to browse the catalog, apply dynamic sorting and filtering to the displayed items, and add selected items to a shopping cart to complete a purchase.
- The system hashes the passwords for the database.

2.2 Use Cases

Use cases describe the interactions between actors and the system in order to accomplish specific tasks.

2.2.1 Use Case 1: Account Authentication

Actor: Visitor

- A visitor interacts with the system to either register a new account or log in to an existing account.
- The system validates the credentials, authenticates the user, and grants the appropriate role permissions (Client or Admin) for the session.

2.2.2 Use Case 2: Browse and Filter Products

Actor: Visitor

- The user navigates the shop interface to view products.
- The user can filter products by category or sort them by attributes such as price or name.
- The system dynamically retrieves and displays the filtered and sorted list of products from the database.

2.2.3 Use Case 3: Cart Management and Checkout

Actor: Client

- An authenticated client adds products to their shopping cart.
- The client reviews the selected items and total price in the cart.
- The client initiates the checkout process to place an order.
- The system processes the order, updates the inventory, and completes the transaction.

2.2.4 Use Case 4: Manage Product Catalog

Actor: Admin

- The administrator accesses the admin panel to manage products.
- The administrator can create new products, edit existing product details (such as price and stock), and delete products from the catalog.
- The system saves the changes in the database and updates the inventory.

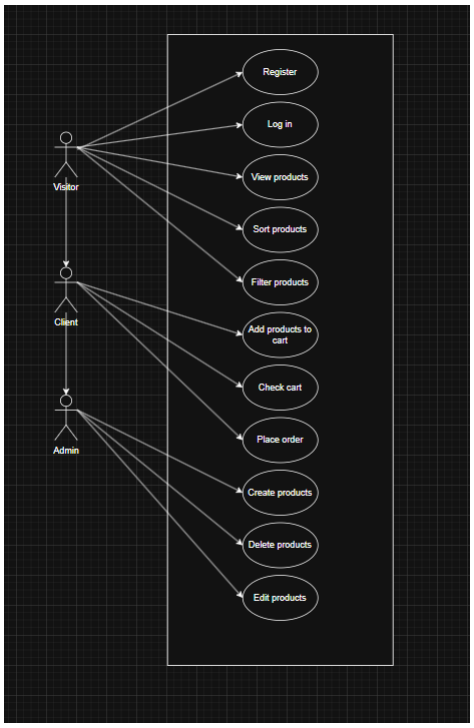

Figure 1: Use Case Diagram

3 Design phase

3.1 Architecture overview

This subsection presents the high-level architecture of the application, which is implemented using the **Model-View-Controller (MVC)** architectural style. This design choice ensures a clear separation between user interface components and the underlying business logic, allowing for modular development and easier maintenance.

3.1.1 System Components

The application is logically divided into three major components:

- **Client Application (View and Controller):** The client-side consists of FXML files for the View and JavaFX Controller classes that handle user interaction. The Controllers are responsible for capturing inputs, such as login attempts or product searches, and delegating the execution to the appropriate Model services.
- **Backend Services (Model):** The backend represents the core logic of the application, encompassing the Service layer and the data entities. These services enforce business rules, such as verifying stock levels before a purchase or validating user credentials during registration.
- **Database (Persistence):** A PostgreSQL database serves as the persistent storage for the Model. Data integrity is maintained through a Repository layer that maps database records to Java entities like Users, Items, and Orders.

3.1.2 Evaluation of the MVC Architectural Style

The implementation of the MVC pattern within this desktop application provides a structured approach to managing data flow.

Advantages:

- **Separation of Concerns:** By decoupling the UI from the business logic, the application becomes significantly easier to debug and extend.
- **Modularity:** Individual components, such as the validation logic in the Service layer, can be modified or tested without affecting the graphical user interface.
- **Data Consistency:** Centralized services, managed via a Service Locator, ensure that all Controllers interact with a single, consistent state of the Model.

Disadvantages:

- **Increased Complexity:** The strict separation between layers requires a higher volume of "boilerplate" code, such as the creation of dedicated Repository and Service interfaces for every entity.
- **Development Overhead:** Implementing even simple features requires synchronized changes across the Controller, Service, and Repository layers, which can slow down the initial development phase.

3.2 Entity-relationship diagram

The database schema is designed to support a robust e-commerce workflow. The following table details the attributes and roles of each entity within the PostgreSQL database.

Figure 2: Entity Diagram

The relationships shown in the diagram ensure that data remains consistent. For instance, the use of foreign keys prevents an item from being assigned to a non-existent category or an order from being placed by an unregistered user.

Entity	Attributes and Responsibilities
`users`	Stores `username`, `password_hash`, `email`, and `role`. Handles user identity.
`categories`	Stores product groupings with `name` and `description`.
`items`	Contains product details, like: `base_price`, `stock_quantity`, and `image_data`.
`orders`	Records the header for a transaction, including `order_date` and `total_price`.
`order_items`	Detailed line items for each order, capturing `quantity` and `price_per_unit`.

Table 1: System Entities and Attribute Descriptions

3.3 Model

The Backend Service Layer acts as the central logic hub of the application, sits within the Model tier of the MVC architecture, and is responsible for enforcing business rules and data integrity.

Responsabilities:

- Business Logic Validation: Ensures that all data operations follow specific constraints, such as verifying that a product name is not empty or that stock levels are sufficient before a purchase.
- State Coordination: Manages transient application states, such as the active shopping cart, and calculates totals dynamically before they are persisted.
- Role-Based Access Control: Restricts specific operations, like deleting items or accessing the admin dashboard, to users with the ADMIN role.

3.4 Controller + View

This represents the View and Controller segments of the MVC pattern, providing the graphical interface through which users interact with the system.

This component serves as the presentation tier. It is decoupled from the database and business logic, meaning it does not execute SQL queries or validate business rules directly; instead, it captures user intent and forwards it to the Service Layer.

Functionalities:

- Dynamic UI Rendering: Uses FXML to define layouts and JavaFX Controllers to populate those layouts with data, such as the product grid in the main shop or the data tables in the admin panel.
- Event Handling: Processes user actions, including button clicks for "Add to Cart," search queries, and navigation between different views like the login screen and the checkout page.
- Visual Feedback: Provides real-time alerts and confirmation dialogs to inform the user of successful actions (e.g., "Item added to cart") or system errors.

- Input Capture: Collects data through forms for product creation, category editing, and user registration, ensuring that raw input is gathered before being sent to the services for validation.

3.5 Design Patterns

Several established software design patterns were utilized during the development of this application to ensure a modular, scalable, and maintainable structure.

- **Model-View-Controller (MVC):** This serves as the primary architectural pattern of the application.
  - *Model:* Represented by the data entities (`Item`, `User`, `Category`) and the services that manage business logic.
  - *View:* Defined through FXML files that describe the graphical user interface layout.
  - *Controller:* Java classes that bridge the interface and the logic by handling user events, such as button clicks or form submissions.
- **Singleton Pattern:** Implemented within the `ServiceLocator` class. This pattern guarantees that a single global instance of the service registry exists throughout the application's execution. The constructor is private, and access is granted exclusively through the static `getInstance()` method, preventing unnecessary memory consumption and state conflicts between services.
- **Repository Pattern:** Used to decouple the business logic from the specific implementation details of the database. By defining a generic interface (`CrudRepository`) and providing specific PostgreSQL implementations, the application remains independent of the underlying storage medium.
- **Dependency Injection:** The application employs dependency injection via constructors or initialization methods. For example, services receive repository implementations through their constructors, which facilitates unit testing and reduces coupling between high-level and low-level components.
- **Strategy Pattern:** In the `MainShopController` through the management of product sorting. The sorting algorithm (by price, name, or category) is selected dynamically at runtime based on the user's selection in the interface, without altering the structure of the object list.

3.6 Class diagrams

Class diagrams describe the static structure of the system by presenting classes, attributes, methods and relationships between classes.

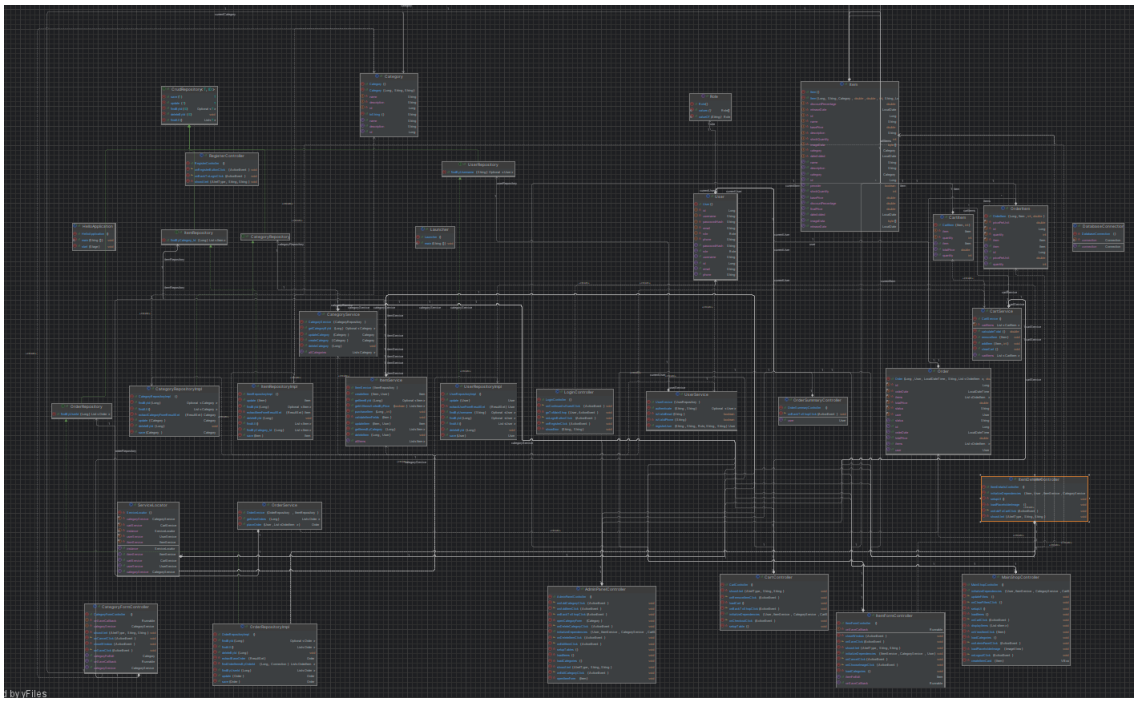

Figure 3: Class Diagram

3.7 Sequence diagrams

Sequence diagrams illustrate how objects interact during the execution of a particular use case.

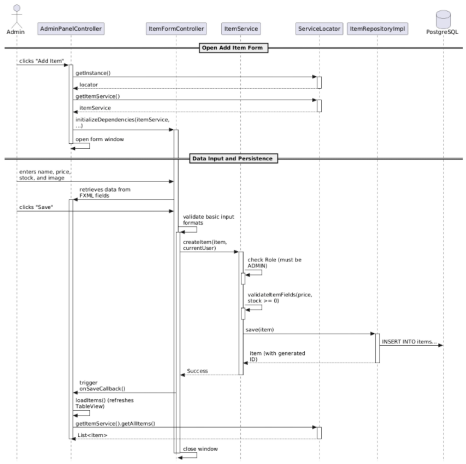

Figure 4: Sequence Diagram - Add New Item as Admin

4 Implementation phase - Application

4.1 Tools and Technologies

This section outlines the technology stack selected for the development of the application. The choices were made to ensure a robust, typed development environment and a reliable persistence layer for e-commerce operations.

4.2 Programming Language: Java

Description: The core application logic is implemented using Java, a high-level, object-oriented programming language.

Reason for Choice: Java was chosen for its strong typing, which reduces runtime errors, and its extensive ecosystem for enterprise-grade applications. It provides the necessary structure for implementing complex design patterns like Singleton and Repository.

4.3 User Interface Framework: JavaFX

Description: JavaFX is used as the primary framework for building the desktop graphical user interface (GUI).

Reason for Choice: It allows for a clean separation of the UI (using FXML) from the application logic (using Controller classes), facilitating the MVC architectural pattern. It also supports CSS-like styling for a modern user experience.

4.4 Database Management System: PostgreSQL

Description: PostgreSQL is used as the Relational Database Management System (RDBMS) to store all persistent data.

Reason for Choice: It was selected for its reliability, support for complex relational queries, and its ability to handle large volumes of data (such as product images stored as byte arrays) with high integrity.

4.5 Frameworks and Third-Party Libraries

- **JDBC (Java Database Connectivity):** Used to establish the connection between the Java application and the PostgreSQL server, enabling SQL execution directly from the Repository layer.
- **BCrypt (jBCrypt):** A critical third-party security library used to hash user passwords before they are stored in the database.

Reason for Choice: Standard plain-text storage is a major security risk; BCrypt provides a salt-based hashing mechanism that is resistant to brute-force attacks.

4.6 Communication Protocols

- **Local Method Invocations:** Since the application is a monolithic desktop client, the Controllers and Services communicate via direct Java method calls within the same Java Virtual Machine (JVM).
- **JDBC Protocol:** The application communicates with the database server using the PostgreSQL JDBC driver protocol over a TCP/IP connection (typically on port 5432).

4.7 Application structure

This project is structured in 3 main packages: model, controller and view. Model itself is also structured in entities, repositories, and services.

4.8 User manual

When you open the app(assuming you have an exe), you can either continue as a guest, register as a new client, or login(if you already have an account).

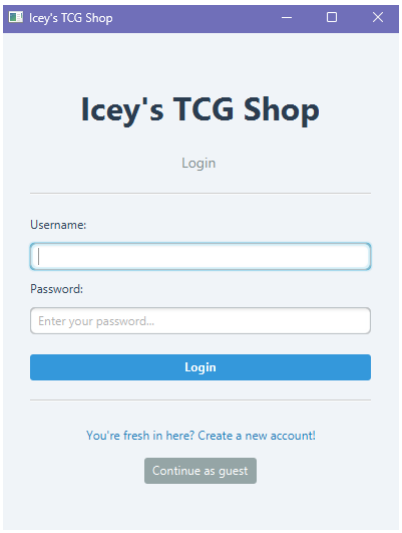

Figure 5: Login Page

The menu is pretty much user friendly, a user can sort or filter the items available in the shop, a user can also check in detail an item and they can add items to the cart. Guests are not allowed to place orders, only Clients and Admins can.

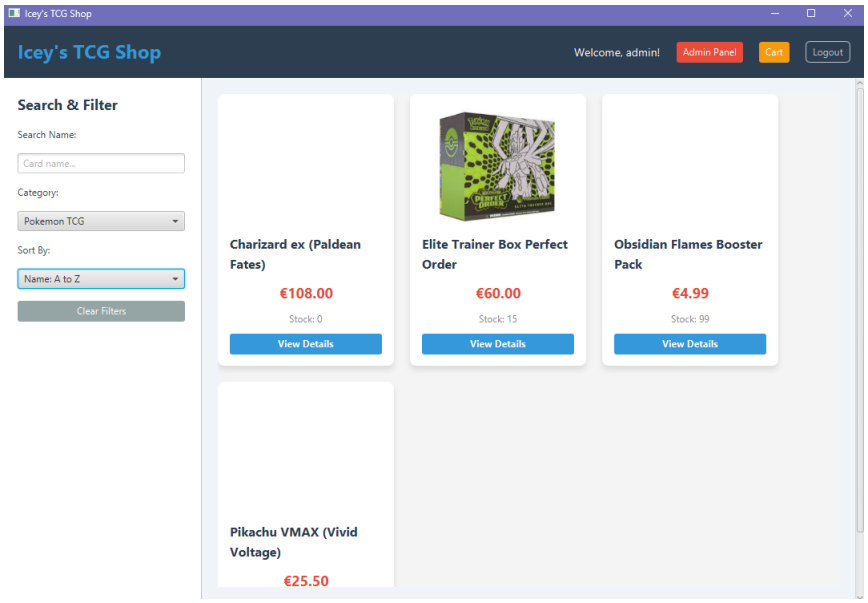

Figure 6: Shop Menu

An Admin can manage items and categories.

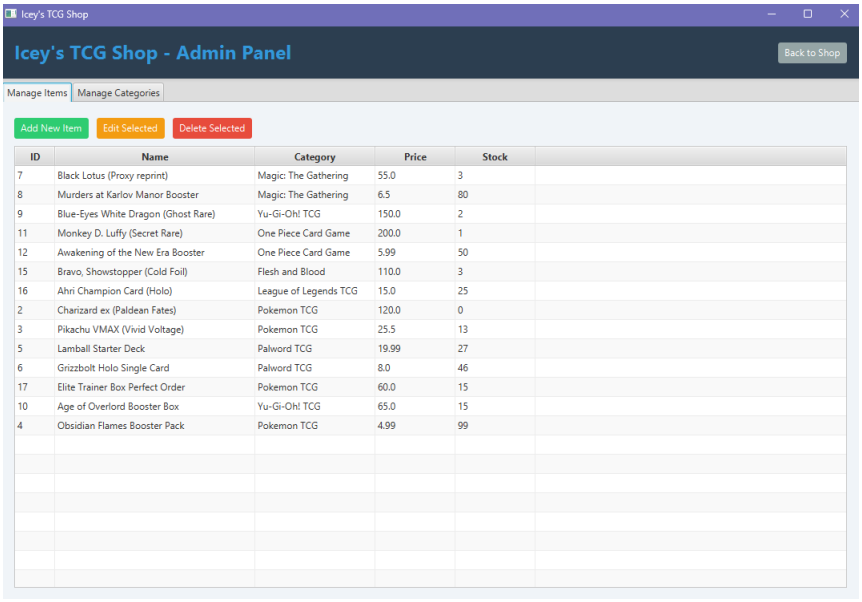

Figure 7: Admin Menu

5 Testing and Validation

This section describes the methods used to verify the correctness of the application and the mechanisms implemented to ensure data integrity.

5.1 Functional Testing

The application underwent manual functional testing to ensure all requirements were met. Key test cases included:

- **Access Control:** Verifying that administrative features are hidden from non-admin users.
- **Transaction Flow:** Ensuring that adding items to the cart and checking out correctly decrements the database stock.
- **Persistence:** Confirming that data persists in the PostgreSQL database after the application is restarted.

5.2 Logic Validation

To prevent system failures, several validation layers were implemented:

- **Input Constraints:** The `ItemService` validates that prices and stock levels are non-negative.
- **Database Integrity:** The `OrderRepository` uses SQL transactions to ensure that orders are either fully completed or not processed at all in case of an error.
- **Format Validation:** Regular expressions are used in the `UserService` to enforce valid email and phone number formats.